Solutions for Linear Data Structures in Real-World Scenarios

LLJ-1

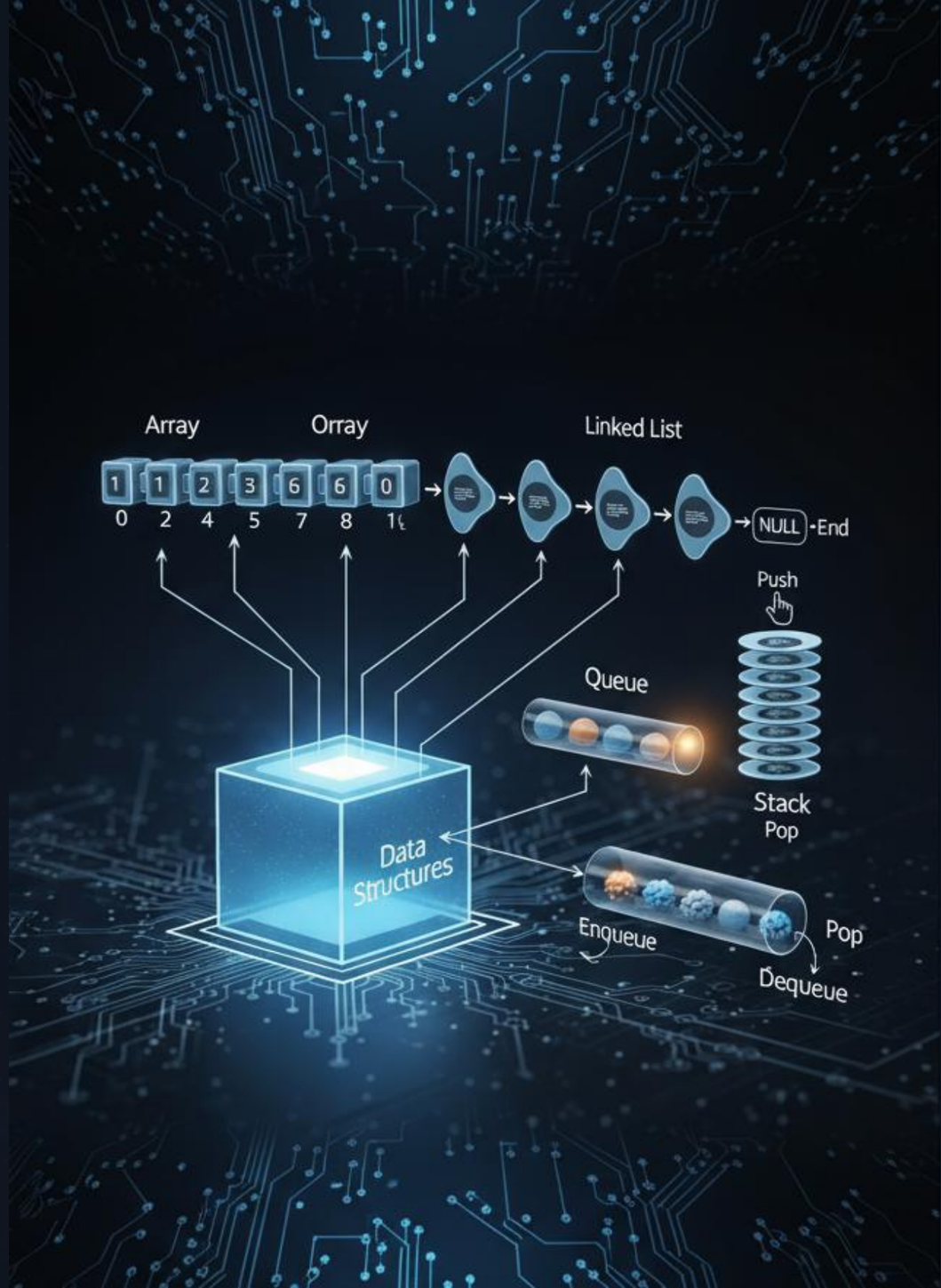
PRESENTED BY:

Sarthak Singh (RA2411026010218)

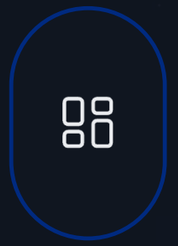
Vartika Jamwal (RA2411026010237)

Smile (RA2411026010223)

AE - 1

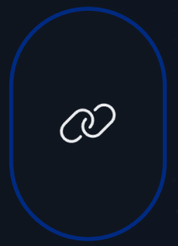


Introduction to Linear Data Structures



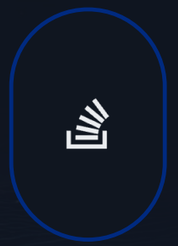
Arrays

A collection of elements stored in contiguous memory locations, accessible by index.



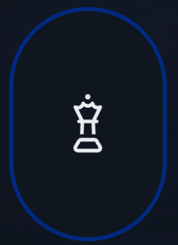
Linked Lists

A sequence of nodes where each node points to the next, allowing dynamic memory allocation.



Stacks

A linear data structure that follows Last-In-First-Out (LIFO) principle.



Queues

A linear data structure that follows First-In-First-Out (FIFO) principle.

Arrays: Usage & Solutions

Real-World Usage

- **Sensor data in IoT**: Arrays store sequential readings from sensors, enabling efficient analysis of environmental data or device performance.
- **Image Processing**: Images are often represented as 2D or 3D arrays (matrices) of pixels, where each element stores color or intensity values.
- **Databases**: Arrays can be used to store collections of data within a single cell, like a list of tags or multiple values associated with an entry.

Solutions

- **Dynamic Arrays**: These arrays automatically grow or shrink in size as elements are added or removed, offering flexibility over fixed-size arrays.
- **Sparse Arrays:** Designed for arrays where most elements are zero or default values, they efficiently store only the non-default elements to save memory.
- **Multidimensional Arrays**: Used to represent data with multiple indices, such as a grid or a table, allowing for complex data structures like matrices or tensors.

Image Processing

```
#include <opencv2/opencv.hpp>
#include <stdio.h>
int main() {
    cv::Mat colorImage =
cv::imread("sample_color_image.jpg");
    if (colorImage.empty()) {
        printf("Could not open or find the image\n");
        return -1;
    }
    cv::Mat grayImage;
    cv::cvtColor(colorImage, grayImage,
cv::COLOR_BGR2GRAY);
    cv::imshow("Original Color Image", colorImage);
    cv::imshow("Grayscale Image", grayImage);
    cv::waitKey(0);
    return 0;
}
```

EXPLANATION:

- An image is represented as a multi-dimensional array of numbers.
- A color image is a 3D array, with dimensions for height, width, and color channels (RGB).
- A grayscale image is a 2D array, with each value representing pixel intensity.
- Using libraries like NumPy, these arrays enable extremely fast, vectorized operations on millions of pixels simultaneously.
- This array-based method is the foundation for modern applications like computer vision, photo editing, and machine learning.

Linked Lists: Usage & Solutions

Real-World Usage

- **Undo-Redo in editors**: Allows efficient tracking of operations for easy reversal or reapplication.
- **Music** Playlist Management: Easily add, remove, and reorder songs without shifting entire data blocks.
- **Blockchain**: Each block contains a hash of the previous block, forming a chain of immutable records.

Solutions

- **Doubly Linked Lists**: Nodes have pointers to both the next and previous elements, enabling bidirectional traversal.
- **Circular Linked Lists**: The last node points back to the first node, creating a continuous loop.
- **Memory** Efficient Nodes: Designed to minimize memory overhead, often by packing data or using specialized pointers.

Undo – Redo in browser and code editors

```
struct Node {
    string state;
    Node* prev;
    Node* next;
    Node(string s) : state(s), prev(nullptr), next(nullptr) {}};
class UndoRedo {
    Node* current;
public:
    UndoRedo(string initial_state) {
        current = new Node(initial_state);}
    void doAction(string new_state) {
        Node* new_node = new Node(new_state);
        new_node->prev = current;
        current->next = new_node;
        current = new_node;}
    string undo() {
        if (current->prev)
            current = current->prev;
        return current->state;}
    string redo() {
        if (current->next)
            current = current->next;
        return current->state;}
    string getState() {
        return current->state;}};
```

EXPLANATION:

- The system uses a doubly linked list, where each node represents a different state of an editor.
- A new node (new state) is created and linked to the current node when an action is performed.
- The undo() function works by moving the system to the previous node, reverting to a past state.
- The redo() function works by moving the system to the next node, reapplying a previously undone action.
- This structure allows for efficient navigation between states, making it a simple way to implement undo/redo functionality.

Stacks: Usage & Solutions

Real-World Usage

- **Browser History**: Stacks keep track of visited pages, allowing users to navigate backward.
- **Expression Evaluation**: Stacks manage operands and operators to correctly process mathematical expressions.
- **Undo Feature**: Stacks store actions in chronological order, making it easy to reverse them.

Solutions

- **Dynamic Stack**: A stack implementation that automatically adjusts its capacity as elements are added or removed.
- **Stack with Min Operation**: A specialized stack that can return the minimum element among its current contents in constant time.
- **Lazy Resizing**: A technique where the underlying array of a stack is resized only when its capacity is fully utilized or falls below a certain threshold.

Expression Evaluation Using Stack

```
int getPrecedence(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    return 0;}
string infixToPostfix(const string& infix) {
    stack<char> st;
    string postfix;
    istringstream tokens(infix);
    string token;
    while (tokens >> token) {if (isdigit(token[0])) {
            postfix += token + ' '; } else if (token[0] == '(') {
            st.push('(');
        } else if (token[0] == ')') {
            while (!st.empty() && st.top() != '(') {
                postfix += st.top();
                postfix += ' ';
                st.pop();}
            if (!st.empty()) st.pop();
        } else { // operator
            while (!st.empty() && getPrecedence(st.top()) >= getPrecedence(token[0])){
                postfix += st.top();
                postfix += ' ';
                st.pop();           }
            st.push(token[0]);      }    }
    while (!st.empty()) {
        postfix += st.top();
        postfix += ' ';
        st.pop();    }
 return postfix;}
```

EXPLANATION:

- The code converts an **infix expression** (e.g., 3+4) to a **postfix expression** (e.g., 3 4+) using a stack.
- **Operands** (numbers) are immediately added to the output expression.
- **Operators** are pushed onto the stack based on their precedence.
- Parentheses control the order of operations:
- An opening parenthesis (is pushed onto the stack.
- A closing parenthesis) triggers the popping of operators from the stack until the matching (is found.
- Any remaining operators on the stack are moved to the output after the input expression is fully processed.
- This ensures the postfix expression correctly reflects the original order of operations, making it easy to evaluate.

Queues: Usage & Solutions

Real-World Usage

- **Print Jobs:** Queues manage the order of documents sent to a printer, ensuring they are processed sequentially.
- **Task Scheduling:** Operating systems use queues to manage processes waiting for CPU time, executing them in the order they arrive.
- **Support Ticketing Systems**: Customer support systems use queues to handle incoming requests, addressing them in the order they are received.

Solutions

- **Circular Queue**: A queue implementation that reuses empty spaces efficiently by connecting the rear to the front, preventing common queue overflow issues.
- **Double-Ended Queue (Deque):** A queue that allows insertion and deletion from both ends, offering more flexibility than a standard queue.
- **Priority Queue**: A special type of queue where each element has a priority, and items with higher priority are processed before items with lower priority.

Real-World Example: CPU Scheduling

Ready Queue

Processes awaiting CPU. This is typically a First-In, First-Out (FIFO) queue, ensuring processes are executed in the order they become ready.

Round-Robin Scheduling

Circular Queue for fair allocation. Processes are given a small time slice, and if they don't complete, they are moved to the end of a circular queue to wait for their next turn.

Multilevel Queue

Priority-based handling. This involves multiple queues, often with different priorities, where processes are moved between queues based on their behavior or urgency, similar to a priority queue system.

CPU scheduling with a FIFO queue

```
int main() {
    queue<string> cpuQueue;
    int n;
    cout << "Enter number of processes: ";
    cin >> n;
    cin.ignore();

    for (int i = 0; i < n; ++i) {
        string process;
        cout << "Enter process name: ";
        getline(cin, process);
        cpuQueue.push(process);
    }

    cout << "\nExecuting processes in order:\n";
    while (!cpuQueue.empty()) {
        cout << "Executing: " << cpuQueue.front() << endl;
        cpuQueue.pop();
    }

    return 0;
}
```

EXPLANATION:

- The program simulates CPU process scheduling using a FIFO (First-In-First-Out) queue.
- The user inputs the number and name of processes.
- Each process is added to the queue in the order it arrives.
- The program then executes processes sequentially by retrieving the process at the front of the queue and removing it after execution.
- This ensures that the first process to arrive is the first to be executed.
- It demonstrates how queues manage tasks in strict arrival order.
- The simulation reflects how a CPU handles ready processes in real operating systems.

Challenges & Solutions

Fixed Array Size

Implement dynamic arrays or linked lists to overcome rigid size limitations and allow for flexible data storage.

Memory Fragmentation

Utilize memory pools or contiguous memory allocation strategies to manage memory more efficiently and reduce fragmentation.

Stack Overflow

Employ dynamic resizing techniques for data structures to prevent overflow issues by automatically adjusting capacity as needed.

Queue Shifting Overhead

Adopt a circular queue implementation to minimize the overhead associated with shifting elements during enqueue and dequeue operations.

Performance Degradation

Optimize data access with efficient indexing and balance data distribution through splitting techniques to improve overall performance.

Conclusion

Simple yet Powerful

Linear data structures are fundamental.

Versatile Applications

Suitable for many real-world scenarios.

Context Matters

Choice depends on use case.

Continuous Innovation

Optimizing usage is ongoing.


THANK YOU